

College of Computing & Informatics (CCI)

SENIOR PROJECT-II REPORT

Family Shield

Author(s):

S210042186	Mohammad Alobaidan
S210047328	Ibrahim Alsanidi
S220037803	Abdulmalik bin Juraybah
S220030456	Mohammed Alsulayyih
S220036864	Mohammed Zain Aldeen
S220022110	Waleed Maashi

Project Supervisor:

Dr. Turki Albaker

Family Shield

By: **Mohammad Alobaidan, Ibrahim Alsanidi, Abdulmalik bin Juraybah, Mohammed Alsulayyih, Mohammed Zain Aldeen, Waleed Maashi**

Thesis/Project submitted to:

College of Computing & Informatics, Saudi Electronic University, Riyadh, Saudi Arabia.

In partial fulfillment of the requirements for the degree of:

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

Project Supervisor

Project Committee Chair

ABSTRACT

In today's digital age, children spend an increasing amount of time online, often without proper supervision or limits. Many parents struggle to manage and monitor their children's internet activities due to the technical complexity of existing router interfaces and the lack of intuitive parental control tools. In this project we tackle this problem by developing a user-friendly router management system designed specifically for parents. The proposed system enables parents to monitor online activity, control access, and block specific online services with a single click without requiring advanced technical knowledge.

The proposed solution focuses on simplicity and accessibility, providing an interface that any parent regardless of technical background can easily navigate. It integrates the core functions of internet filtering, access control, and activity monitoring into a single, intuitive dashboard. The system is built using flexible, open technologies that can be adapted or replaced as needed, ensuring long-term scalability. Its design philosophy emphasizes clear visual feedback, one-click actions, and minimal setup, making it an approachable tool for the everyday household rather than a system meant for technical users.

We hope this project addresses a real-world problem and sparks innovation in protecting younger generations, highlighting issues that have often been overlooked due to the evolving nature of digital platforms. While popular platforms offer basic parental filters, they cannot always guarantee that all content is appropriate or safe for children. For this reason, we aim for our project to regard routers as the first line of defense, providing parents with a simple, intuitive, and effective tool to safeguard children in the cyber world.

DEDICATION

1. Dr. Turki Albaker (For his guidance)
2. Dr. Hassan Alqahtani (For his Insight)
3. Dr. Waleed Aldosire (For his knowledge)

PREFACE

This project report presents the development of a user-friendly router system designed to help parents monitor and control their children's online activities. The purpose of this preface is to provide clarifications regarding the scope and approach of the project. The focus of this work is to create a simple, practical solution that addresses a real-world problem which is the need for accessible and effective parental controls in the digital age. This report emphasizes the conceptual design, system implementation, and evaluation of a prototype, rather than the production of a commercial-grade product.

The project combines knowledge from networking, web development, user interface design, as well as cybersecurity defense to demonstrate how technology can be used to enhance online safety for children. Throughout the report, we have aimed to maintain clarity and accessibility so that both technical and non-technical readers can understand the methodology and findings. Any assumptions, limitations, or design decisions made during the project have been explained in relevant sections to ensure transparency.

We would like to express our sincere gratitude to our project supervisor, for his guidance, support, and valuable feedback throughout this work. We also extend our heartfelt thanks to our families, especially our parents, for their continuous encouragement and hands-on support during testing and providing feedback. As representatives of the demographic this project is aimed at, their contributions were instrumental in ensuring the success of this work.

REVISION HISTORY

Name	Date	Reason For Changes	Version
Mohammad Alobaidan & Mohammad Zain Aldeen	8.10.2025	Ch.1 Introduction	V.1
Mohammad Alobaidan & Mohammad Zain Aldeen	9.10.2025	Ch.2 Literature Review	V.2
Waleed Maashi & Mohammed Alsulayyih	22.10.2025	Ch.3 Methodology	V.3
Waleed Maashi & Mohammed Alsulayyih	04.11.2025	Ch.3 Methodology (Update)	V.3.1
Ibrahim Alsanidi & Abdulmalik bin Juraybah	12.11.2025	Ch.4 System Analysis	V4
Mohammad Zain Aldeen	19.11.2025	Ch5. System Design	V.5
Mohammad Alobaidan	20.11.2025	Ch6. Discussion & Conclusion	V.6
Waleed Maashi	24.11.2025	Added Lists of Tables & Figures	V.7

TABLE OF CONTENTS

<i>LIST OF FIGURES</i>	7
<i>LIST OF TABLES</i>	7
<i>CHAPTER 1: INTRODUCTION</i>	8
1.1 Project Background/Overview:	8
1.2 Problem Description:	8
1.3 Project Scope:	9
1.4 Project Objectives:	9
1.5 Project Structure/Plan:	10
<i>CHAPTER 2: LITERATURE REVIEW</i>	14
<i>CHAPTER 3: METHODOLOGY</i>	26
<i>CHAPTER 4: SYSTEM ANALYSIS</i>	32
4.1 Product Features:	32
4.2 Functional Requirements:	32
4.3 Nonfunctional Requirements.....	38
4.4 Analysis Models	39
<i>CHAPTER 5: SYSTEM DESIGN</i>	40
<i>CHAPTER 6: DISCUSSION & CONCLUSION</i>	45
6.1 Discussion	45
6.2 Conclusion	46
<i>REFERENCES</i>	48
<i>APPENDIX: Glossary</i>	49

LIST OF FIGURES

Figure 1 Project Gantt Chart	11
Figure 2 ADM/PDM Diagram	12
Figure 3 Mind Map.....	13
Figure 4 AdGaurd Home	15
Figure 5 NextDNS	16
Figure 6 SafeDNS.....	17
Figure 7 CleanBrowsing.....	18
Figure 8 TP-Link HomeCare.....	19
Figure 9 Linksys Velop.....	20
Figure 10 Circle Home Plus	21
Figure 11 Qustodio.....	22
Figure 12Firewalla.....	23
Figure 13 Synology	24
Figure 14 Family Shield's dashboard.....	25
Figure 15 Use Case Diagram	40
Figure 16 Component Diagram	41
Figure 17 Deployment Diagram	42
Figure 18 Design Level Sequence Diagram	43
Figure 19 Entity-Relationship Diagram	44
Figure 20 sequence diagram.....	45

LIST OF TABLES

Table 1 Use-Case 1	33
Table 2 Use-Case 2	34
Table 3 Use-Case 3	35
Table 4 Use-Case 4.....	36
Table 5 Use-Case 5	37

CHAPTER 1: INTRODUCTION

The increasing presence of the internet in everyday life has transformed how children access information, communicate, and entertain themselves. While online platforms offer educational and recreational opportunities, they also expose children to content that may be inappropriate or unsafe. Parents often struggle to monitor and control this exposure due to the technical complexity of network devices and parental control tools. This project aims to address this gap by developing a simple, user-friendly router-based system that allows parents to monitor online activity, block specific services, and protect their children in a non-intrusive, intuitive way.

1.1 Project Background/Overview:

With the increasing use of the internet by children, parents face significant challenges in monitoring and controlling online activities. Popular social media and video platforms provide basic parental controls, but these are often insufficient, as they cannot ensure that all content is safe or age appropriate. Many parents find existing router interfaces and parental control tools too complex to configure.

This project proposes a user-friendly router-based system aimed at simplifying parental control, providing monitoring, and enabling one-click blocking of specific online services. The focus is on ease of use for everyday parents, ensuring that non-technical users can effectively supervise their children's online activity. The system integrates simple internet filtering, usage monitoring, and control features into a single dashboard, making the router a central tool in online safety.

1.2 Problem Description:

The rapid growth of online content accessible to children presents a risk: platforms' built-in parental filters cannot guarantee safe or age-appropriate experiences. Parents often lack the

technical knowledge to configure advanced network settings, leaving children exposed to unsuitable content.

There is a clear need for a simple, intuitive solution that empowers parents to:

- Monitor which services and websites their children access.
- Block inappropriate content or services with minimal effort.
- Apply these controls consistently across all devices connected to the home network, both wired and wireless.

1.3 Project Scope:

- Designing and implementing a dashboard that provides parents with a visual overview of connected devices and internet usage.
- Developing an easy-to-use control system for blocking or restricting access to selected online services, or to the network as a whole.
- Ensuring compatibility with both Wi-Fi and Ethernet devices connected to the router.
- Providing activity logging and basic reporting to help parents track usage patterns.

Benefits:

- Simplifies online safety management for parents with minimal technical knowledge.
- Demonstrates how a standard router can act as the first line of defense against unsuitable content.
- Provides a foundation for further innovation in family-focused network security.

Expected Outcome:

- A working prototype of a user-friendly parental control system integrated with a router interface.
- A functional dashboard showing device activity, service usage, one-click blocking, and more.

1.4 Project Objectives:

1. To develop a simple, intuitive dashboard for parents to monitor and control online activity.
2. To enable one-click blocking of selected online services and platforms.
3. To enforce scheduled restrictions on internet usage.
4. To provide usage monitoring and logging for both Wi-Fi and Ethernet devices.
5. To demonstrate the feasibility of integrating parental control features directly into a router system.
6. To produce a prototype that is easily deployable and understandable by non-technical users.

1.5 Project Structure/Plan:

1. Project Plan:

The project begins by studying how parents currently manage their children's internet access and identifying limitations in existing solutions. Data is collected through short surveys and online research to understand the most important features (such as ease of use, blocking, and monitoring).

2. System Design:

Once requirements are gathered, the system's structure is planned. This includes designing the interface layout for the dashboard, defining data flow, and determining how the router interacts with the control system. The emphasis during this phase is on **simplicity and user experience**.

3. Prototype Development:

The core functionality is implemented during this stage. The dashboard is developed, service-blocking logic is configured, and device monitoring is integrated. The goal is to produce a working prototype that demonstrates the main concept clearly.

4. Testing and Evaluation:

The system is tested using various devices connected via Wi-Fi and Ethernet. Parents

(the target demographic) provide feedback on usability and effectiveness. Any issues discovered are refined before final documentation.

5. Documentation and Reporting:

The final phase involves compiling the report, including technical explanations, testing results, and user feedback. Supporting materials such as diagrams, screenshots, and graphs are added to demonstrate project outcomes.

- This Gantt Chart is a timeline that represents the rough estimate of the weekly period needed to complete each section of the project.

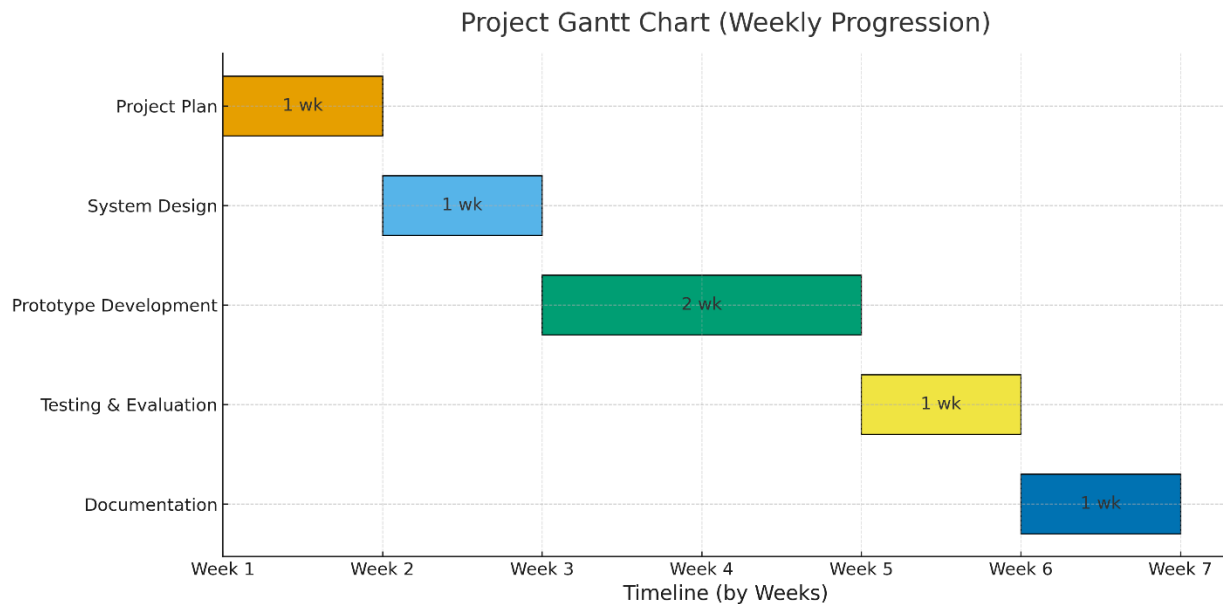


Figure 1 Project Gantt Chart

- This ADM/PDM Diagram is a representation of the progression order for each phase needed to accomplish the entire project.

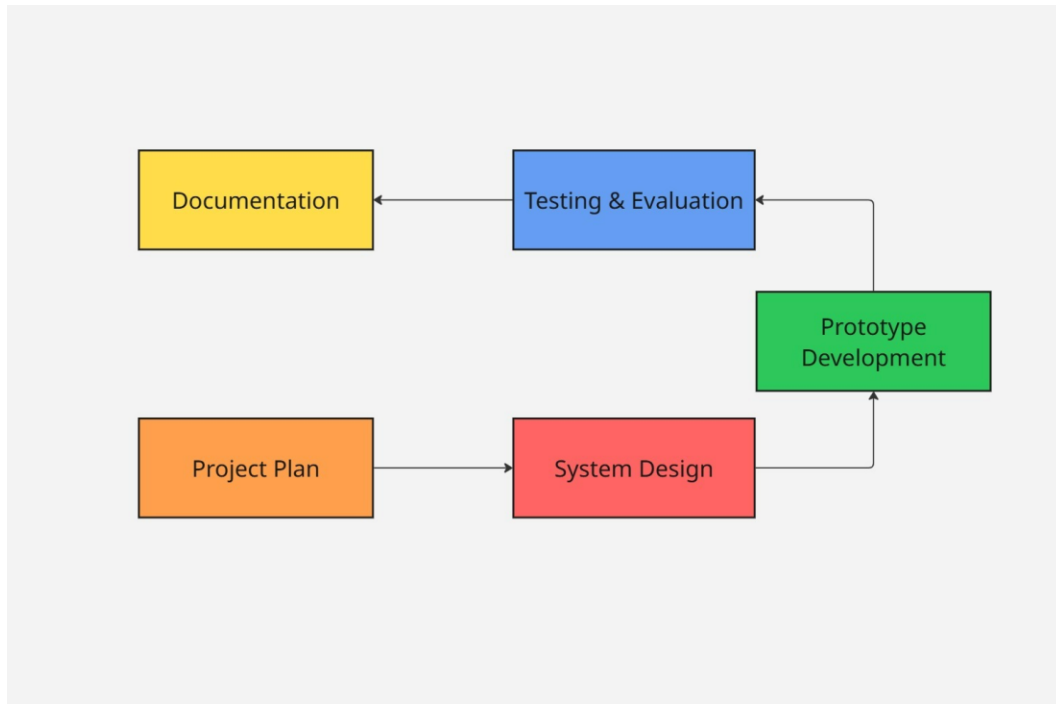


Figure 2 ADM/PDM Diagram

- This Mind Map illustrates the collaborative effort between each of the six students involved in this project at handling each phase as a group, showing the division of responsibilities as well as the flexibility needed to meet our collective goal.

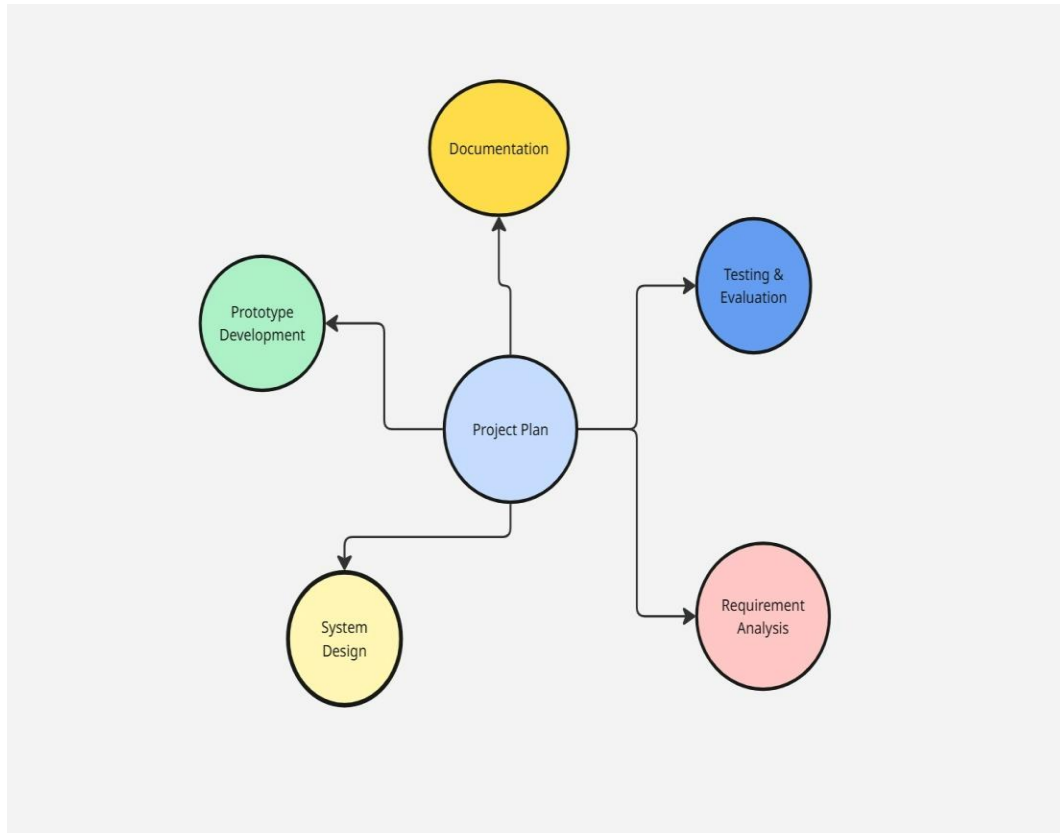


Figure 3 Mind Map

CHAPTER 2: LITERATURE REVIEW

This chapter compares the proposed project, a router-based, parent-friendly system that allows monitoring and one-click blocking of social media, to existing commercial parental-control products, DNS/ router-level services, and relevant HCI/networking research. The goal is to identify gaps where current offerings either require technical setup, push subscriptions, or offer controls that are not simple enough for an everyday parent. The comparison supports the motivation for the proposed router prototype and helps define the design requirements (simplicity, robust blocking, cross-device consistency, and clear reporting).

Commercial products (hardware) and services (software) comparison:

1. Pi-hole with AdGuard Home (Our inspiration)



Figure 4 AdGaurd Home

What it does: Local DNS-level filtering system (blocks unwanted sites and ads before they load, at the network level), can be configured for content blocking.

Strengths: Free and network-wide; customizable rules.

Limitations: Requires extra hardware such as a Raspberry Pi or a home server; setup complexity makes it unsuitable for most parents who lack technical knowledge.

2. NextDNS

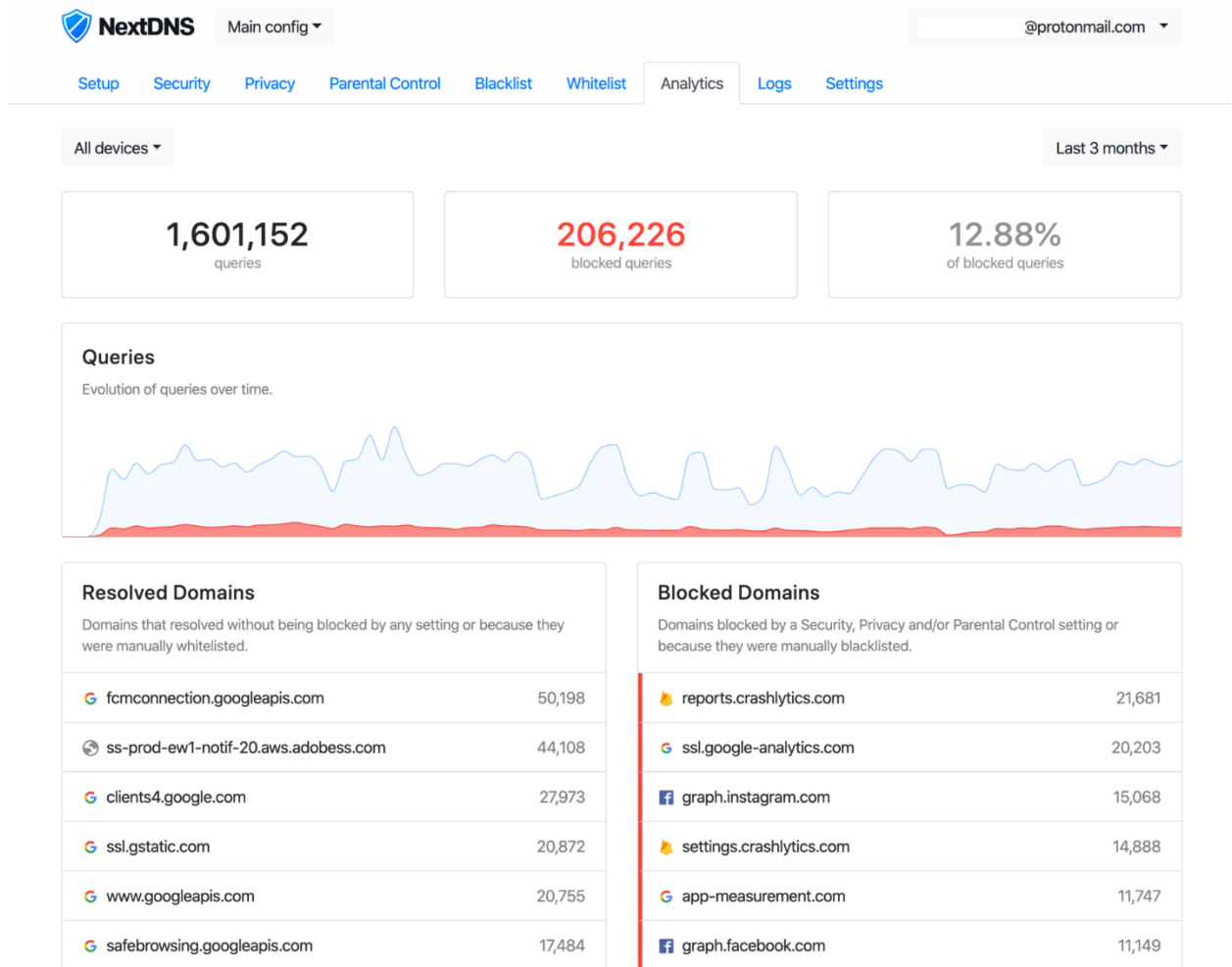


Figure 5 NextDNS

What it does: Cloud-based DNS that offers customizable filtering and analytics.

Strengths: Highly customizable; supports DoH, DoT, and DNSCrypt; detailed logs and privacy features; works across all devices and platforms.

Limitations: Requires setup per device or router; filtering limited to DNS level (cannot block content within apps or encrypted traffic).

3. SafeDNS

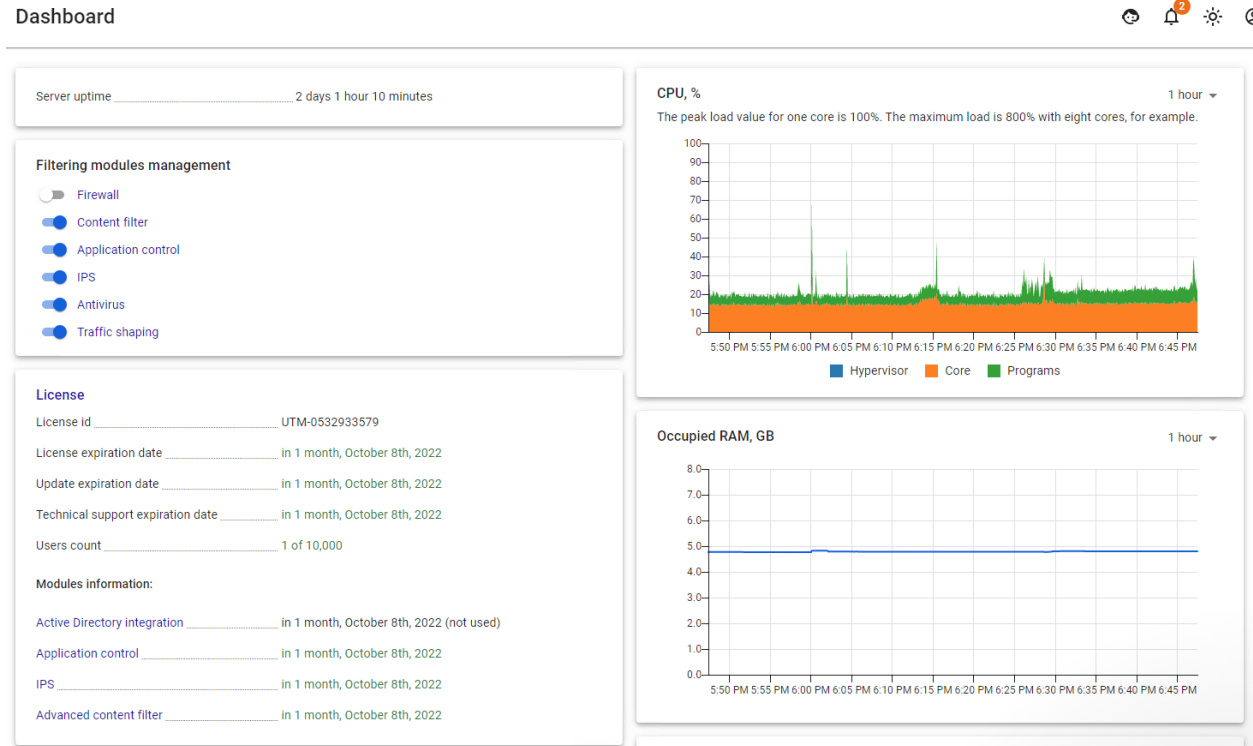


Figure 6 SafeDNS

What it does: Cloud-based DNS filtering service for families and schools, offering categories, whitelists, and time restrictions.

Strengths: Easy to use; detailed reports; can protect all devices on a network.

Limitations: Subscription-based and DNS-only filtering (blocks unwanted sites and ads before they load, at the network level), cannot block apps.

4. CleanBrowsing

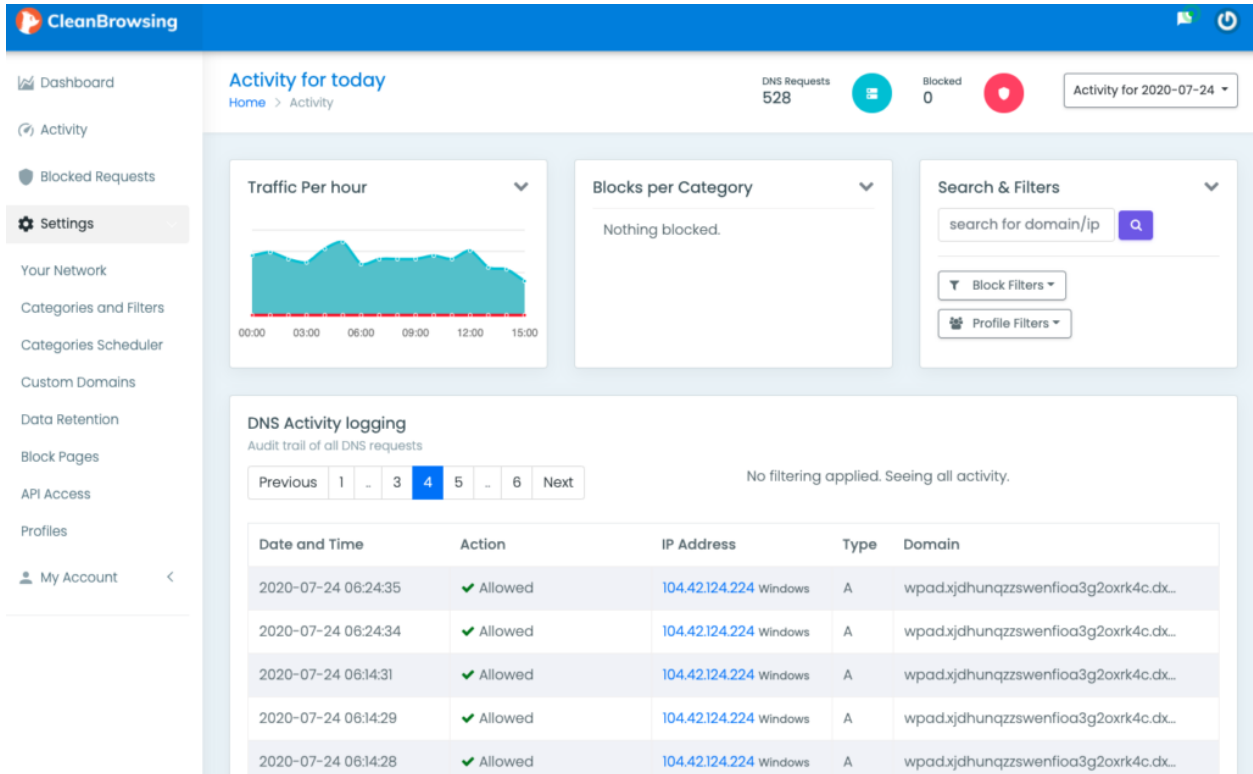


Figure 7 CleanBrowsing

What it does: Provides free and paid list-based family filtering that blocks adult, violent, or malicious sites.

Strengths: Easy setup; privacy-focused; no account required for the free tier.

Limitations: DNS-based filtering only (blocks unwanted sites and ads before they load, at the network level). cannot block specific apps or social media platforms due to encrypted traffic.

5. TP-Link HomeCare

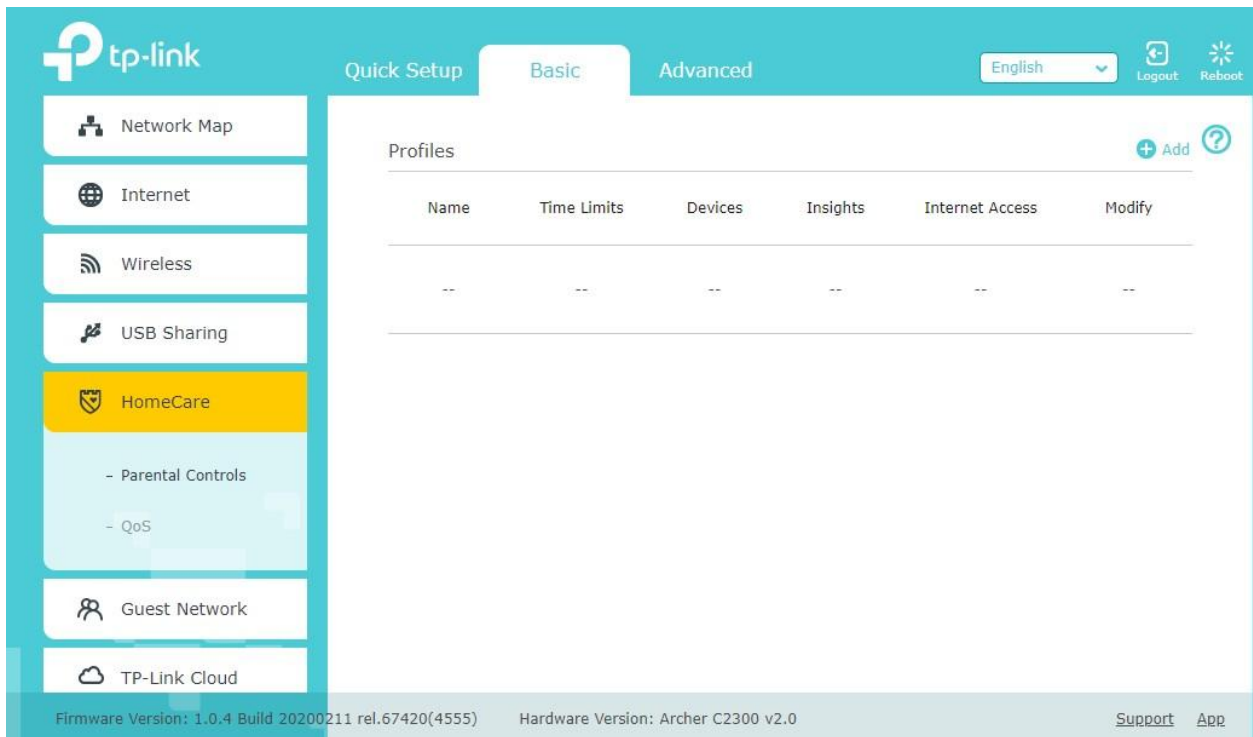


Figure 8 TP-Link HomeCare

What it does: Built into TP-Link routers, offering category filters, bedtimes, and device pauses.

Strengths: Works natively within the router app; simple scheduling.

Limitations: Lacks one-click app/service blocking and fine-grained visibility of traffic activity.

6. Linksys Velop

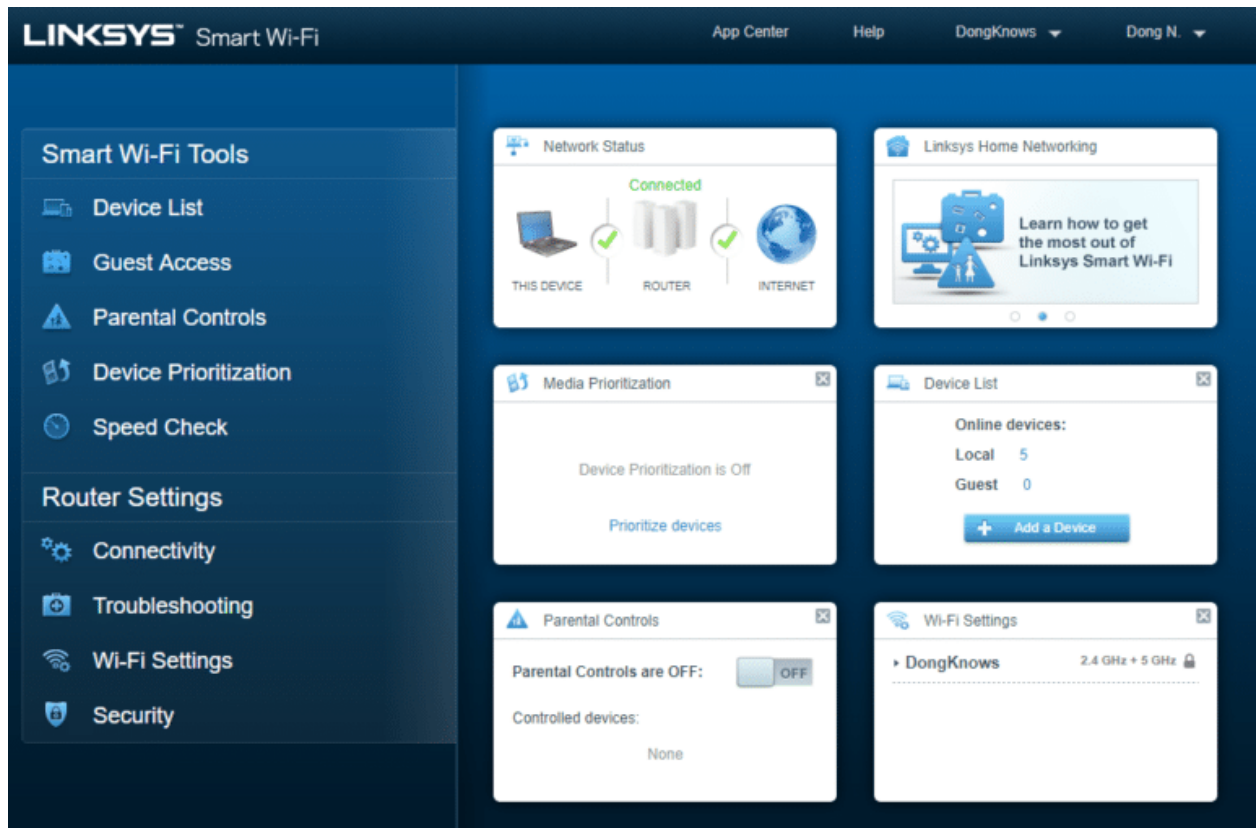


Figure 9 Linksys Velop

What it does: Linksys Smart WiFi app enables parents to schedule internet access, block specific websites, and pause devices.

Strengths: Simple mobile interface; works across all devices on the network.

Limitations: Limited filtering categories; lacks one-click social media blocking.

7. Circle Home Plus

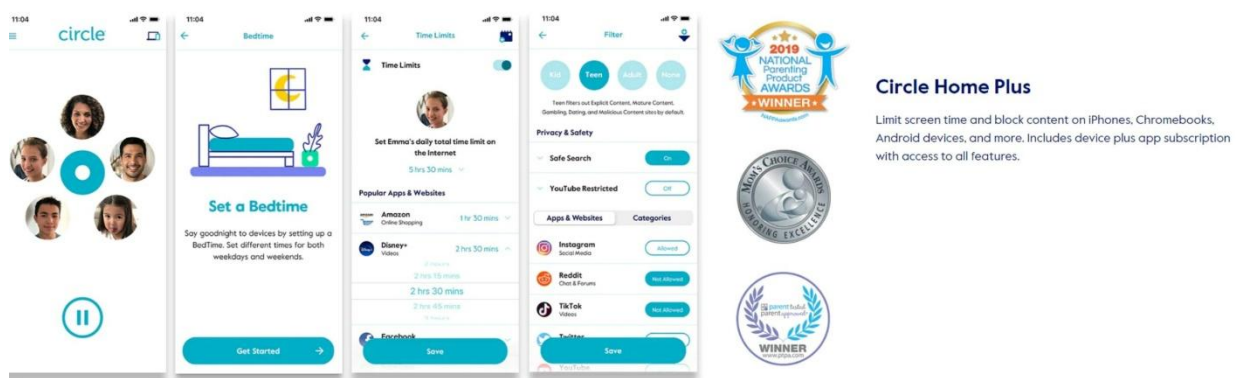


Figure 10 Circle Home Plus

What it does: Connects to existing routers to manage screen time, filter content, and pause the internet per device.

Strengths: Designed for parents with simple per-device control and profiles.

Limitations: Requires extra hardware and occasional technical maintenance.

8. Qustodio

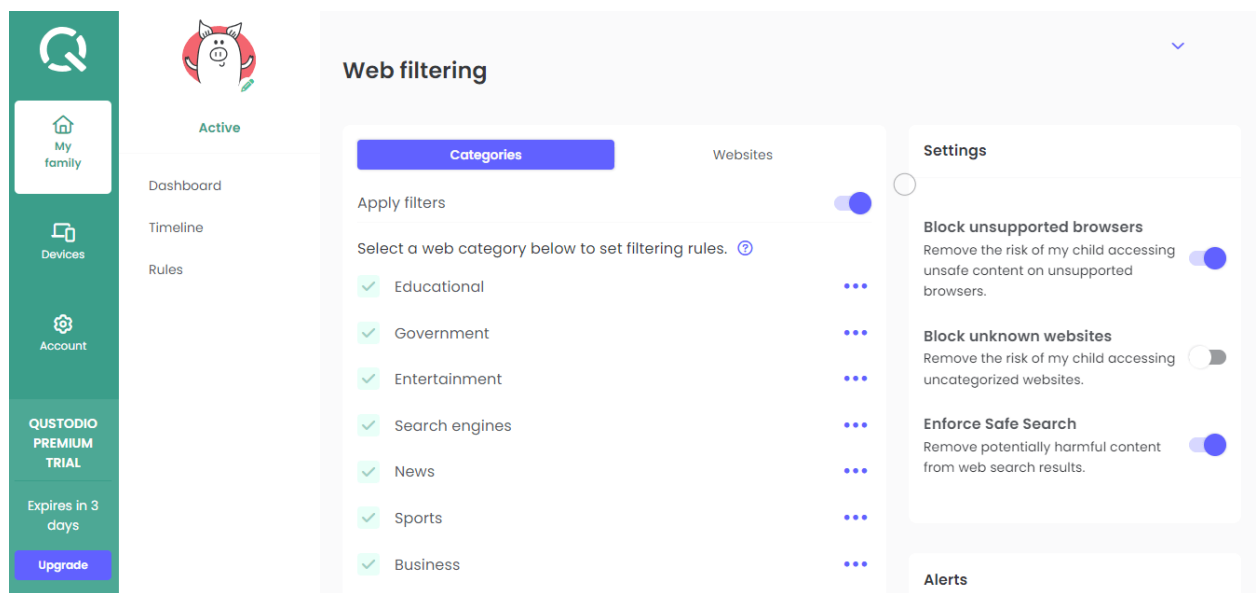


Figure 11 Qustodio

What it does: Offers app blocking, screen-time limits, and reports via a cloud dashboard for all devices.

Strengths: Clear, user-friendly interface and detailed activity summaries.

Limitations: Requires installation on each device; can be bypassed with mobile data or VPNs. Not integrated at the router or gateway level.

9. Firewalla

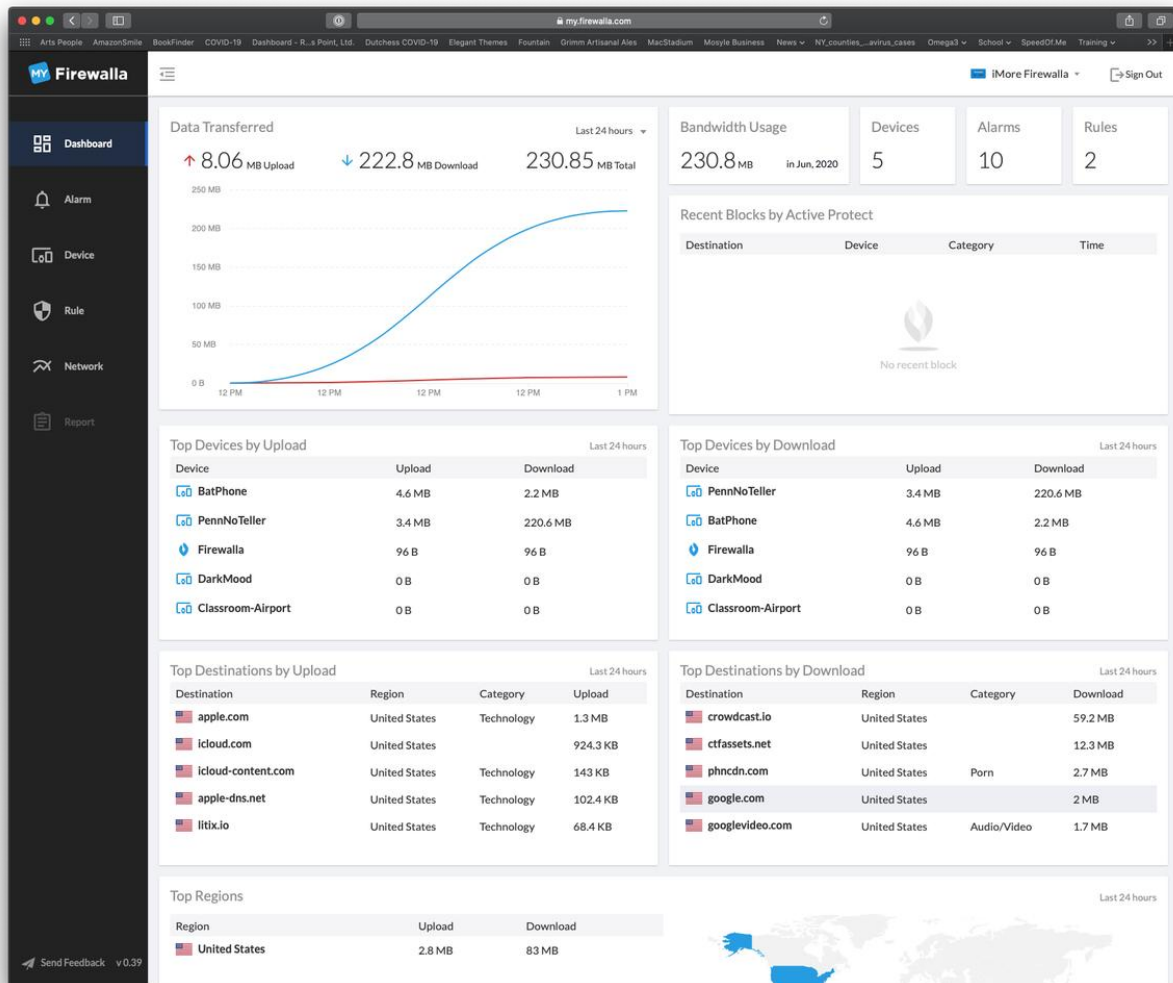


Figure 12Firewalla

What it does: A small hardware unit that connects to home networks for real-time monitoring, ad-blocking, and parental controls.

Strengths: Network-level protection for all wired and wireless devices.

Limitations: Setup may still be complex for non-technical users. Advanced features can overwhelm average parents.

10. Synology

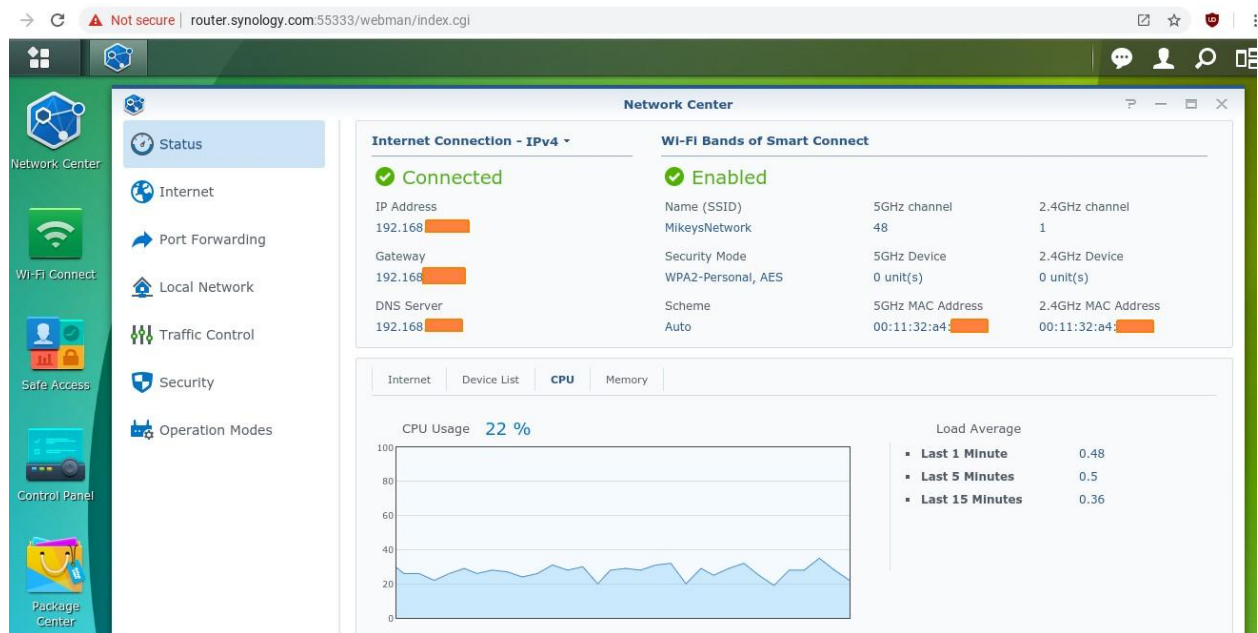


Figure 13 Synology

What it does: A small hardware unit that connects to home networks for real-time monitoring.

Strength: Network-level protection for all wired and wireless devices, not just one device.

Limitations: Need specific hardware, setup may still be complex for non-technical users. Advanced features may overwhelm parents.

Here is our preliminary vision of the dashboard used to control the **Family Shield** router.

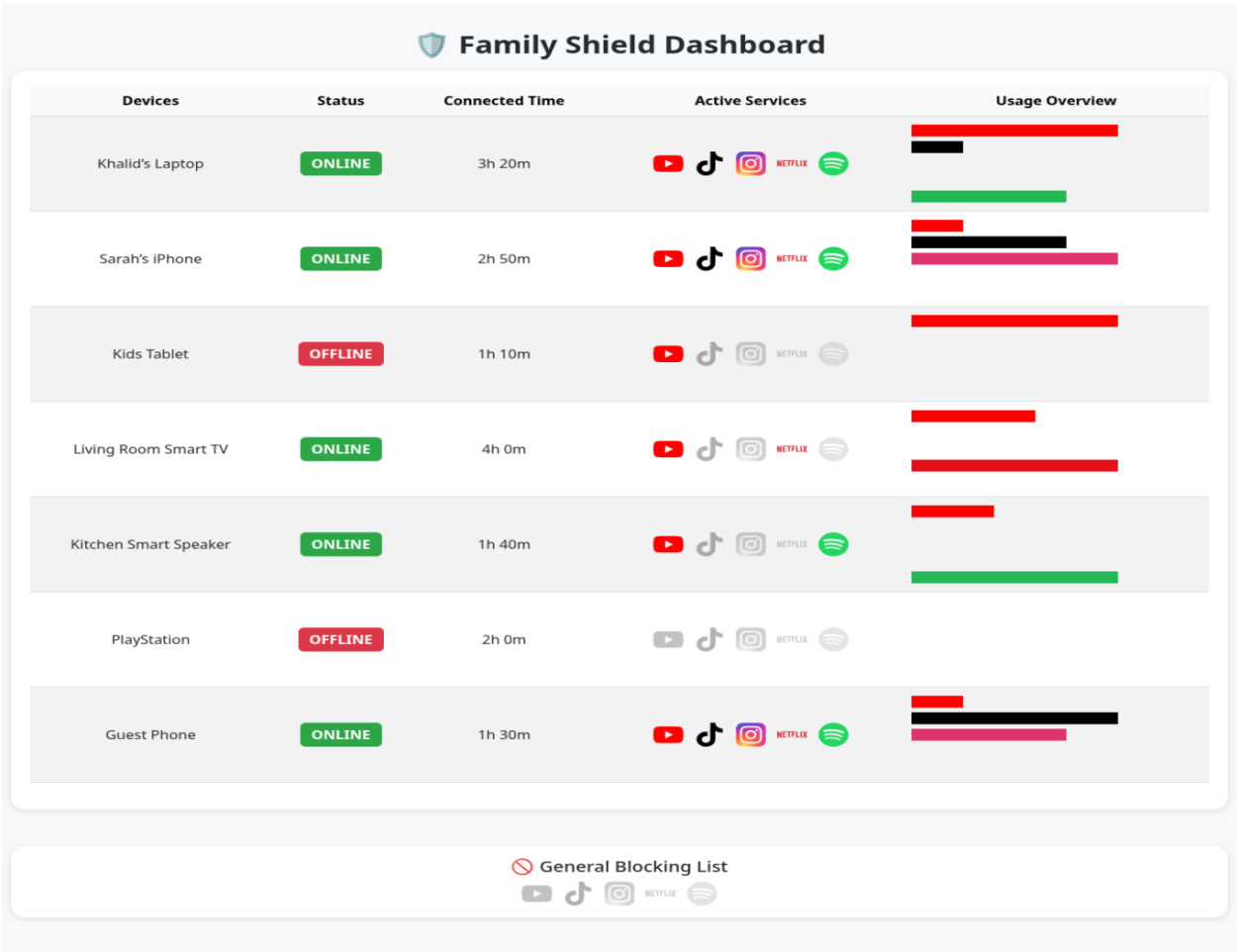


Figure 14 Family Shield's dashboard

As you can see the dashboard for our router contains the following:

1. A simple, clear, intuitive view that anyone regardless of their technical prowess can use with no hurdles.
2. Devices names while abstracting technical terms like IP/MAC addresses, also only show these devices only while connected to make it less cluttered and visually simple to grasp.
3. A status toggle that can cut internet access all together in a single click.
4. Time since the device connected to the router to monitor the time spent online.

5. A block list containing popular platforms that can be blocked for each device or globally for all devices with the General Blocking List at the bottom, also in a single click.
6. An overview section which shows the platforms being used to enhance monitoring.

CHAPTER 3: METHODOLOGY

3.1 Agile background

Agile is an iterative approach that delivers value in small increments rather than a single large launch. Teams review requirements and results frequently, accept change when needed, and test throughout the development cycle to maintain quality. These practices help produce a working slice at regular intervals, which supports early feedback and course correction (Ashtari, 2022). The approach aligns with the values and principles of the Agile Manifesto that emphasize individuals and interactions, working software, collaboration, and responding to change (Beck et al., 2001). When implemented through Scrum, Agile uses a lightweight framework of roles, events, and artifacts to organize short delivery cycles known as sprints (time-boxed iterations) (Schwaber & Sutherland, 2020).

3.2 Rationale for using Agile on Family Shield

Family Shield must remain simple for parents while adapting to changing services and network behavior. New platforms and bypass methods can appear at any time. Agile supports this context through frequent delivery, focused feedback, and controlled adjustments between sprints (time-boxed iterations) (Ashtari, 2022). During reviews, the team validates wording and flows with testers (evaluation participants) while keeping parents as the primary audience.

3.3 Process and ceremonies

We operate two-week sprints (time-boxed iterations) with four standard events, following common Scrum guidance (Schwaber & Sutherland, 2020).

- **Sprint planning**
Select the highest-value items from the product backlog, define acceptance criteria, and confirm test devices and data.
- **Daily checkups**
A brief checkpoint to surface blockers and align work across network, backend, frontend, and QA (Quality Assurance).
- **Sprint review**
Demonstrate the working slice on the OpenWrt router to stakeholders and testers (evaluation participants), capture structured feedback, and confirm visible value for parents.
- **Retrospective**
Record specific improvements for the next sprint (time-boxed iteration) such as clearer labels, faster flows, or stronger test coverage.

3.4 Sprint plan

The plan progresses from setup to reporting in small, testable steps, using clear wording per supervisor guidance (Ashtari, 2022).

- **Sprint 0 — Setup**
Prepare the OpenWrt router. Initialize the Flask project. Create the SQLite (Structured Query Language Lite) database. Connect basic DNS (Domain Name System) blocking. Establish version control and continuous integration.
- **Sprint 1 — Devices and per-device internet pause**
Implement device discovery. Show friendly device names, online or offline status, and connected time. Add a per-device internet pause. Deliver server-rendered HTML (HyperText Markup Language) pages through Flask.
- **Sprint 2 — Service blocking v1**
Provide one-click toggles for socials as an example TikTok, YouTube, and Instagram at

device level and global level. Ensure rules persist after router reboot. Record actions in an audit log stored in SQLite.

- Sprint 3 — Profiles and time controls
Create child profiles that group devices. Add bedtime and homework windows. Apply rules through profile inheritance and resolve basic conflicts.
- Sprint 4 — Monitoring and reports
Show an Active Services overview. Produce daily and weekly usage summaries. Export reports to CSV and PDF. Render simple charts on the server and embed them in HTML.

3.5 Roles and responsibilities

This is one cross-functional student team. Role definitions are adapted from Scrum's lightweight guidance (Schwaber & Sutherland, 2020).

- Product Owner
Prioritizes value for parents, maintains the backlog, and approves wording in the dashboard.
- Scrum Master
Facilitates the process, removes impediments, and tracks action items from retrospectives.
- Network and backend
Integrate with OpenWrt. Implement DNS (Domain Name System) and basic DPI (Deep Packet Inspection) policy paths. Build the policy engine. Expose Flask routes. Design SQLite models.
- Frontend and UX (User Experience)
Produce server-rendered HTML (HyperText Markup Language) pages with clear forms and tables. Ensure accessibility. Support report rendering.
- QA (Quality Assurance)
Define test plans. Manage a device matrix. Run acceptance tests. Document results.

3.6 Architecture and technical method/tools

- Platform
An existing router with OpenWrt acts as the gateway on a small test network in the.
- Backend
Python with Flask for pages and internal API (Application Programming Interface) routes. Use simple blueprints and a clear folder layout to help new team members onboard quickly.
- Frontend
Server-rendered HTML (HyperText Markup Language) templates. Forms and tables with readable labels. Focus on accessibility and simple navigation.
- Database
SQLite (Structured Query Language Lite) for configuration, audit events, schedules, and usage counts. Store a single database file with the project, and provide a brief guide to reset and seed the data.
- Policy enforcement path
Start with DNS (Domain Name System) block lists and small curated domain or IP sets. Add basic DPI (Deep Packet Inspection) only when feasible on the router. Enforce router DNS and optionally block common DoH (DNS over HTTPS) endpoints to reduce bypass.
- Test devices and data
Use laptops and phones on Wi-Fi and Ethernet. Maintain a small device matrix and sample usage data so demonstrations are repeatable.
- Performance and limits
The router has limited CPU and RAM. Keep rule sets small. Measure simple timings. Document limits clearly in the report.
- Security and privacy for a class setting
Do not store packet payloads. Store only device names, profile links, policy states, timestamps, and aggregate counters. Admin access is local. Change default passwords during setup.

- Reproducibility
Provide a step-by-step setup guide and version-controlled code. Tag releases for each sprint (time-boxed iteration) to show progress to the instructor.
- Ethics and scope
This is an academic prototype that demonstrates one-click service blocking and schedules. It is not a commercial parental control product. The report states what it can and cannot do.

3.7 Definition of Done

A backlog item is complete when all of the following are met. The feature works on the router and survives reboot when relevant. Policy apply and rollback complete in about three seconds when relevant. Unit and integration tests pass in continuous integration. UI (User Interface) pages meet accessibility checks including labels, contrast, and keyboard navigation. A short help note is added to the user guide. The sprint (time-boxed iteration) review demo succeeds on real devices. These criteria reflect Agile guidance that working software is the primary measure of progress and that teams reflect and adjust regularly (Beck et al., 2001; Schwaber & Sutherland, 2020).

3.8 Quality assurance and evaluation

The testing approach follows Agile emphasis on continuous verification and regular delivery of working software (Ashtari, 2022; Beck et al., 2001).

- Unit tests
Schedule evaluator, policy compilation, and database access.
- Integration tests
DNS (Domain Name System) and basic DPI (Deep Packet Inspection) classification.
Firewall rule application on OpenWrt. Flask to SQLite data flow.
- End-to-end tests
Block a named service on a device and verify the effect. Set a bedtime window and verify enforcement. Generate and download a weekly report.

- Usability checks
Structured sessions with testers (evaluation participants) each sprint (time-boxed iteration) to confirm that labels are clear and flows are easy to follow.
- Performance targets
First HTML render within two seconds on a local network. Policy propagation within three seconds where applicable.

3.9 Risks and mitigations

- DNS-only coverage gaps
Some applications may not be fully blocked. Use a layered approach that combines DNS (Domain Name System) lists with selected IP ranges and basic DPI (Deep Packet Inspection) when possible. Display a brief note in the interface that explains limits.
- Bypass through VPN or custom DNS
Enforce router DNS (Domain Name System) and optionally block common DoH (DNS over HTTPS) endpoints. Display a warning when a bypass is detected.
- Over-blocking
Provide allow lists and a quick undo. Record actions in SQLite (Structured Query Language Lite) to support traceability.
- Router resource limits
Keep rule sets small. Monitor CPU (Central Processing Unit) and RAM (Random Access Memory) usage. Optimize lists and signatures.
- Unclear wording
Use simple labels. Confirm them with testers (evaluation participants) in each sprint (time-boxed iteration) review.

3.10 Deliverables and timeline

- Hardware and operating system
OpenWrt router.

- Working application
Flask backend. Server-rendered HTML (HyperText Markup Language). SQLite (Structured Query Language Lite) database.
- MVP (Minimum Viable Product) features
Device list. Per-device pause. One-click service blocking at device and global level. Profiles. Schedules. Usage summaries. CSV (Comma Separated Values) and PDF (Portable Document Format) export.
- Demonstration
Live service toggle and schedule enforcement on real devices at the end of Sprint 4.
- Documentation
Setup guide. Test results. Screenshots. Known limits. A plan for future enhancements.

CHAPTER 4: SYSTEM ANALYSIS

4.1 Product Features:

- User-friendly parental control dashboard accessible via router web interface.
- Monitoring connected devices and their internet activity.
- One-click blocking for specific websites or online services.
- Time-based restrictions for children's internet usage.
- Real-time usage reports and activity logging.
- Compatibility with both Wi-Fi and Ethernet devices.

4.2 Functional Requirements:

Use-Case 1

Table 1 Use-Case 1

Identifier		UC-1
Purpose		Display a list of all devices connected to the router with their status and activity.
Priority		High
Pre-conditions		Router and dashboard are online.
Post-conditions		Device list and activity logs are updated.
Typical Course of Action		
S#	Actor Action	System Response
1	Parents log into the dashboard.	System verifies credentials and opens the main dashboard.
2	Parents select “View Devices.”	System displays all connected devices with names and IPs.
3	Parent clicks a device name.	System shows usage stats and activity history.
Alternate Course of Action		
S#	Actor Action	System Response
1	Parents try to access the dashboard but enter incorrect credentials.	System displays an error message: “Invalid username or password.”
2	Parents retry or select “Forgot Password.”	System provides password recovery options.
3	Parents attempt to view devices while router is offline.	System displays a message: “No devices found, check router connection.”

Use-Case 2

Table 2 Use-Case 2

Identifier		UC-2
Purpose		Allow parents to block access to specific sites or apps.
Priority		High
Pre-conditions		Dashboard is active, and devices are connected.
Post-conditions		Selected service is blocked across all devices.
Typical Course of Action		
S#	Actor Action	System Response
1	Parents open “Service Control.”	System loads list of common platforms (YouTube, TikTok, etc.).
2	Parents toggle “Block” for a service.	System applies firewall rules to deny access.
3	Parents confirm change.	System updates the rules and displays a confirmation message.
Alternate Course of Action		
S#	Actor Action	System Response
1	Parents select a service that is already blocked.	System displays a notice: “Service already blocked.”
2	Parents try to block a service while offline.	System shows error: “Network unavailable please reconnect.”
3	Parents cancel action before confirmation.	System discards the change and returns to the main dashboard.

Use-Case 3

Table 3 Use-Case 3

Identifier		UC-3
Purpose		Allow parents to schedule internet access restrictions for specific times.
Priority		Medium
Pre-conditions		Dashboard is active, and at least one device is connected.
Post-conditions		Selected devices follow the defined access schedule.
Typical Course of Action		
S#	Actor Action	System Response
1	Parents open "Schedule Control."	System loads current device list and existing schedules.
2	Parents select a device and set allowed internet hours.	System validates time input and stores schedule settings.
3	Parents confirm schedule.	System updates router rules and shows confirmation message.
Alternate Course of Action		
S#	Actor Action	System Response
1	Parents enter overlapping or invalid times.	System displays an error: "Invalid schedule time."
2	Parents cancel scheduling before saving.	System discards changes and returns to main dashboard.

Use-Case 4

Table 4 Use-Case 4

Identifier		UC-4
Purpose		Provide activity logs and usage reports for all connected devices.
Priority		Medium
Pre-conditions		Devices have been connected and activity has been recorded.
Post-conditions		Parents can view and export summarized activity data.
Typical Course of Action		
S#	Actor Action	System Response
1	Parents open “Activity Log.”	System retrieves and displays recent usage data for all devices.
2	Parents filter by date or device.	System updates the log view with selected filters.
3	Parents select “Export Report.”	System generates a downloadable summary file.
Alternate Course of Action		
S#	Actor Action	System Response
1	Parents open the log when no data is available.	System displays: “No activity recorded.”
2	Parents try to export before data is loaded.	System shows message: “Please wait, loading data.”

Use-Case 5

Table 5 Use-Case 5

Identifier		UC-5
Purpose		Allow parents to pause or resume internet access for all devices with one click.
Priority		High
Pre-conditions		Router and dashboard are online, and devices are connected.
Post-conditions		Network access is paused or restored according to user action.
Typical Course of Action		
S#	Actor Action	System Response
1	Parents open the dashboard and click “Pause Internet.”	System disables internet access across all devices.
2	Parents confirm the action.	System applies the change and shows “Internet Paused” status.
3	Parents click “Resume Internet.”	System restores connectivity and updates status.
Alternate Course of Action		
S#	Actor Action	System Response
1	Parents try to pause internet while offline.	System shows error: “Action unavailable, please reconnect.”
2	Parents cancel before confirming.	System keeps network access unchanged.

4.3 Nonfunctional Requirements

Performance Requirements

- The system must load the dashboard within 3 seconds.
- Changes (blocking a service) should take effect within 5 seconds.
- Must support at least 10 active devices simultaneously without performance issues.

Safety Requirements

- Prevent accidental blocking of essential network services (e.g., DNS, router access).
- Include confirmation prompts before applying major restrictions.
- System should fail safely, ensuring network recovery in case of errors.

Security Requirements

- Require user authentication for dashboard access.
- Use HTTPS encryption for communication between the dashboard and router.
- Maintain local logging of actions (block/unblock, login) for accountability.

Software Quality Attributes

- **Usability:** Interface must be intuitive for non-technical users.
- **Reliability:** System should run continuously without unexpected restarts.
- **Maintainability:** Modular design allows updates to filtering lists.
- **Portability:** Should operate across routers with similar firmware platforms.

Other Requirements

- Compliance with family-friendly internet filtering guidelines.
- Multilingual interface (English/Arabic).
- Logs are exportable in CSV or PDF format for reports.

4.4 Analysis Models

The Use-Case Diagram below illustrates the main interactions between the Parent, Router System, and Child Device.

It provides an overview of how the parent uses the router's dashboard to monitor and control children's online activities through functions such as authentication, monitoring, blocking, scheduling, and viewing activity logs.

- The **Parent** interacts directly with all system functions through the router dashboard.
- The **Router System** executes commands and manages network access for all connected devices.
- The **Child Device** represents any device connected to the router that is affected by parental rules (phones, tablets, laptops).
- The **View Activity Logs** use case allows both the Parent and Router System to access recorded activity information for review.

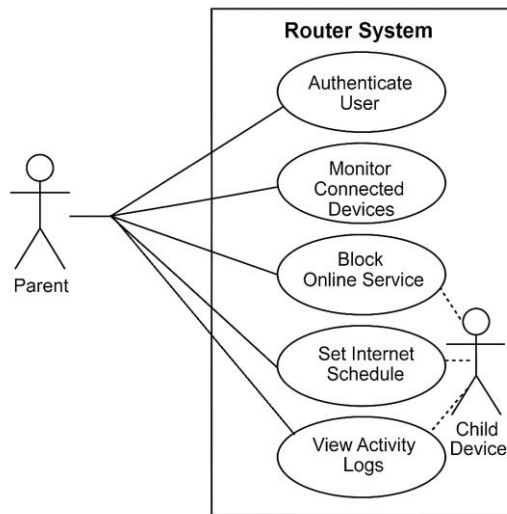


Figure 15 Use Case Diagram

CHAPTER 5: SYSTEM DESIGN

5-1 Component Diagram

The component diagram illustrates the main building blocks of the **Family Shield** system and their interactions. Key components include the dashboard, policy engine, device discovery service, database, and scheduler. This diagram provides a clear overview of the system's structure and shows how each part works together to deliver parental-control features efficiently.

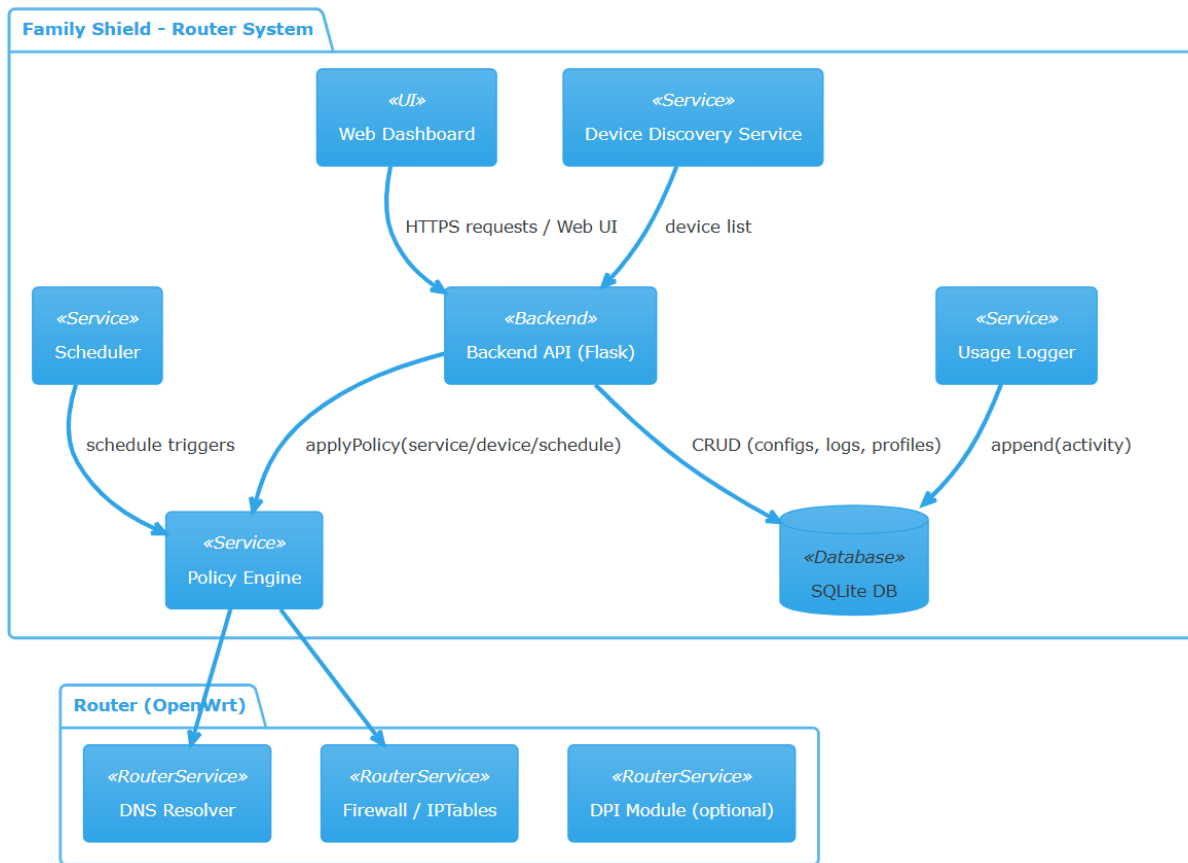


Figure 16 Component Diagram

5-2 Deployment Diagram

The deployment diagram shows how the system's software components are distributed across hardware. It highlights the **OpenWrt** router, the backend server built with **Flask**, the **SQLite** database, and the connected devices. This diagram helps visualize how the system operates in a real environment and how all components communicate to ensure proper monitoring and control.

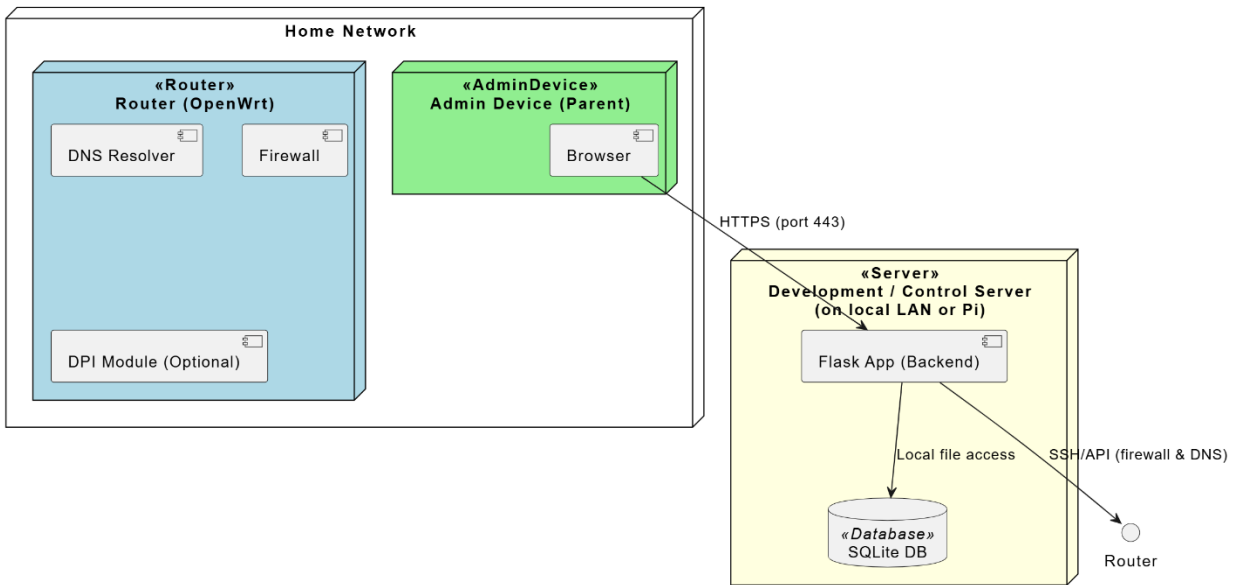


Figure 17 Deployment Diagram

5-3 Design Level Sequence Diagram

The design-level sequence diagram illustrates how a parent interacts with the system and its components during normal use. It shows the sequence of actions such as logging in, viewing

connected devices, blocking services, scheduling internet access, and checking usage reports. The diagram clarifies the flow of interactions and the system's responses to each action.

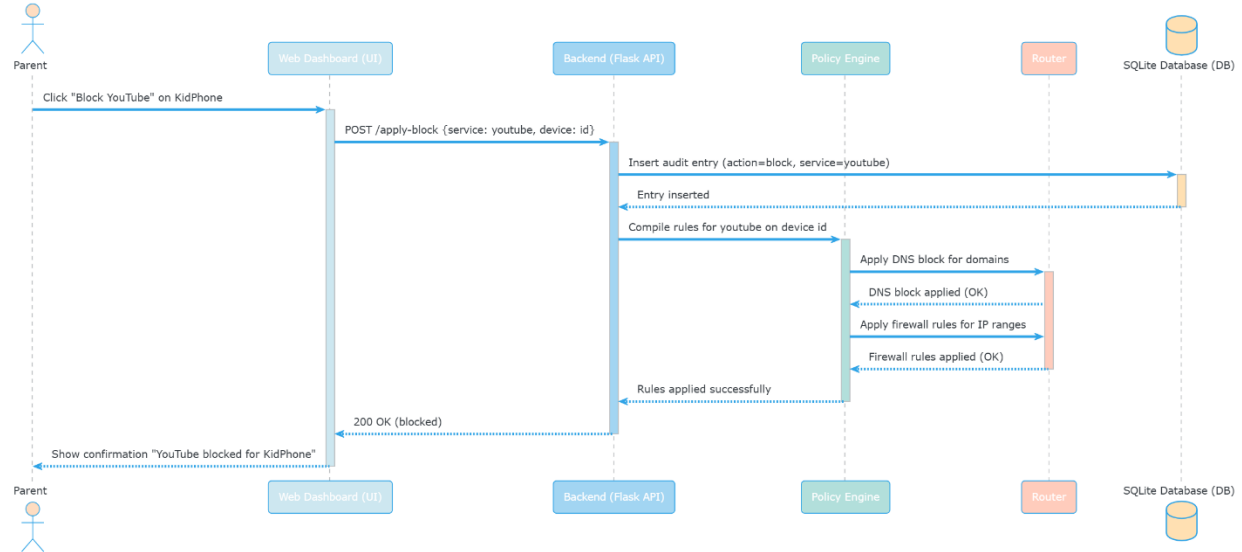


Figure 18 Design Level Sequence Diagram

5-4 Entity-Relationship Diagram

The Entity-Relationship Diagram (ERD) presents the structure of the system's database. Main entities include:

- **Devices:** connected devices on the network.
- **Profiles:** child profiles that define rules for devices.
- **Services:** platforms that can be blocked or monitored.
- **Schedules:** allow internet times for devices or profiles.
- **Activity Logs:** logs of device activity on the network.

The diagram shows the relationships between entities, such as which devices belong to which profiles and how activity is recorded. It provides a clear blueprint for database management and tracking internet usage.

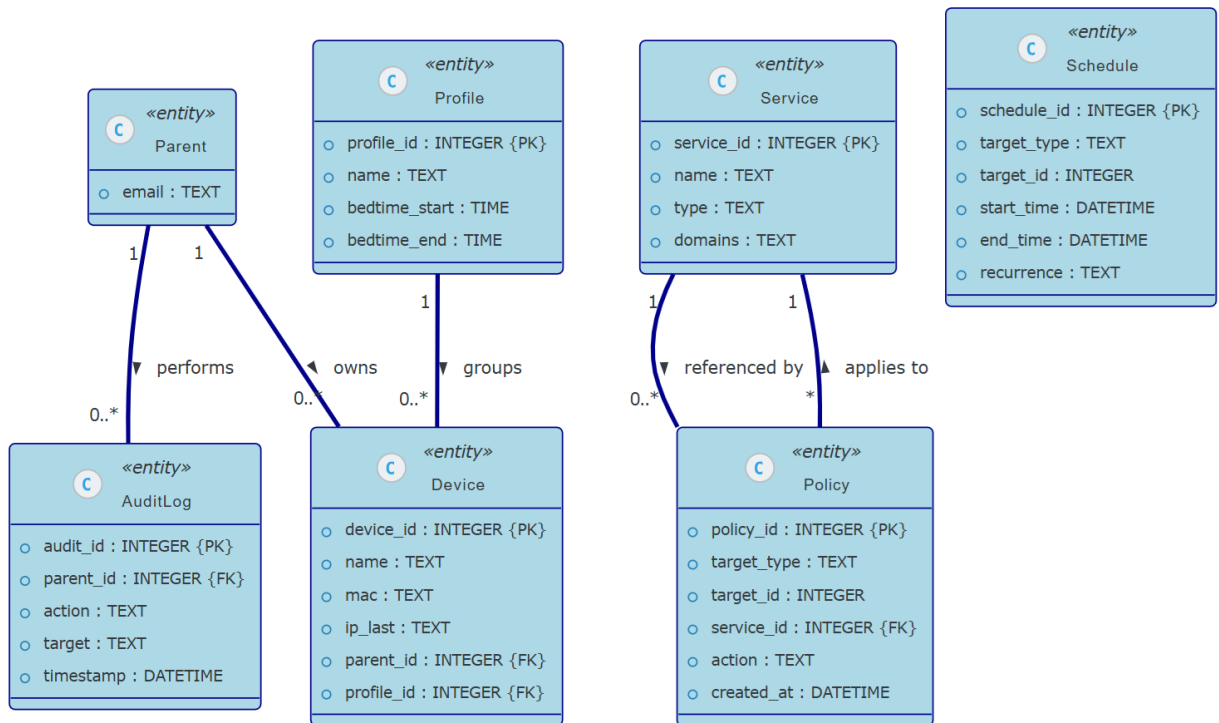


Figure 19 Entity-Relationship Diagram

5-5 Sequence Diagram

The sequence diagram demonstrates the flow of interactions over time for specific system actions. For example, when a parent blocks a service, the dashboard sends the request to the policy engine, which then updates the router to enforce the block. The system logs the action and sends a confirmation back to the parent. This diagram helps visualize step-by-step operations and communication between components.

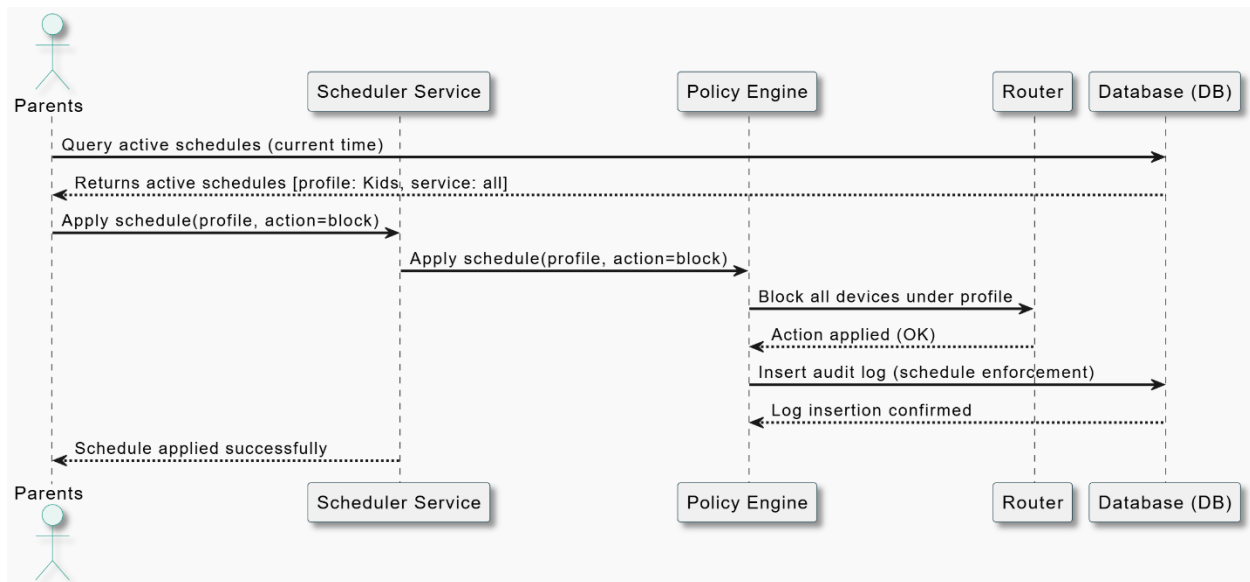


Figure 20 sequence diagram

CHAPTER 6: DISCUSSION & CONCLUSION

6.1 Discussion

The work carried out in this project reflects a broader challenge many families face today: children now engage with digital platforms more than ever, yet the tools available to parents often feel overly technical or fragmented. Our review of existing solutions ranging from list-based services to commercial hardware showed that most tools either require technical familiarity, involve additional costs, or fail to provide consistent protection across all devices in the home. These observations reinforced the central idea behind Family Shield: parental controls are only truly effective when they are simple enough for everyday users to understand and use confidently.

Throughout the development process, the Agile approach allowed the system to evolve in response to feedback and practical testing. Short sprints, combined with input from parents, highlighted the importance of small but meaningful design decisions: clear labeling, uncomplicated device listings, and actions that take only a single click. These iterations also made it clear how essential it is to balance functionality with the limitations of consumer-grade routers. By relying primarily on list filtering supplemented with basic packet control when possible we were able to implement practical service blocking without overwhelming the router's limited processing resources.

At the same time, the project revealed the inherent limits of router-level parental controls. Some applications, especially those using encrypted traffic, remain difficult to block fully, and determined users may still attempt to bypass restrictions with VPNs or custom DNS configurations. Rather than overlooking these issues, the project addresses them openly and offers reasonable mitigation strategies, such as enforcing router DNS and blocking common DoH endpoints. These reflections underline an important academic point: even when the technology has shortcomings, thoughtful design and clear communication can still make a system valuable and reliable within its intended scope.

6.2 Conclusion

Family Shield demonstrates that parental-control tools do not need to be complicated to be effective. By redesigning the router interface around clarity and ease of use, the project delivers a

prototype that allows parents to supervise their children's online activity with minimal technical effort. The dashboard, device monitoring, one-click service blocking, and scheduling features work together to show what a more approachable form of home network management can look like. In doing so, the project offers a practical response to a growing need in modern households.

From a technical standpoint, the project also confirms that open-source platforms such as OpenWrt, Flask, and SQLite provide a solid foundation for building lightweight yet capable network-management tools. Despite the constraints of router hardware, the system succeeds in demonstrating real-time control and meaningful activity reporting. While the prototype is not positioned as a commercial product, it opens the door to several promising directions for future work such as stronger traffic classification, mobile interfaces, cloud-based synchronization, and improved VPN detection. These enhancements could significantly expand the system's capabilities without sacrificing the simplicity that defines its core identity.

Ultimately, Family Shield takes a practical and user-centered approach to a challenge that continues to grow as children spend more time online. By treating the router as the first line of defense and prioritizing usability, the project contributes to ongoing efforts to make digital spaces safer for younger users. It stands as a foundation on which more advanced solutions can be built, and as a reminder that effective technology often begins with thoughtful, human-centered design.

REFERENCES

Ashtari, H. (2022, September 7). *What is Agile software development? Life cycle, methodology, and examples*. Spiceworks. <https://www.spiceworks.com/tech/devops/articles/what-is-agile-software-development>

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for Agile software development*. <https://agilemanifesto.org/>

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. Scrum Guides. <https://scrumguides.org/>

APPENDIX: Glossary

A

- **Agile**
A software development approach that delivers work in small increments and adapts to change through continuous feedback.
- **API (Application Programming Interface)**
A set of rules that allow different software components to communicate with each other.
- **Authentication**
The process of verifying a user's identity before allowing access to the system.

B

- **Backend**
The part of the system that handles logic, data processing, communication with the router, and internal operations.

C

- **CSV (Comma-Separated Values)**
A file format used to export data in a simple table structure, viewable in Excel and similar programs.
- **CPU (Central Processing Unit)**
The main processing component of a device or router that executes instructions.

D

- **DPI (Deep Packet Inspection)**

A technique that examines internet traffic to identify or classify the type of service being used.

- **DNS (Domain Name System)**

A system that converts website names which humans understand into numerical IP addresses for the machine to understand.

- **DoH (DNS over HTTPS)**

A secure method of sending DNS requests using encrypted HTTPS connections.

E

- **Ethernet**

A wired connection type used to connect devices to the router through a physical cable.

F

- **Firewall**

A network security tool that allows or blocks traffic based on predefined rules.

- **Flask**

A Python-based web framework used to build the router's control dashboard.

- **Frontend**

The part of the system that users see and interact with (web pages, buttons, labels).

H

- **HTML (HyperText Markup Language)**

The standard language used to create the structure of web pages.

O

- **OpenWrt (Open Wireless Router)**

A customizable open-source operating system used on the router to enable advanced features.

P

- **PDF (Portable Document Format)**

a versatile file format that preserves a document's appearance, formatting, and fonts across different devices and operating systems.

Q

- **QA (Quality Assurance)**

Processes that test the system to ensure it works correctly and meets requirements.

R

- **RAM (Random Access Memory)**

Short-term memory in the router used to run processes and store active data.

- **Router**

The device that connects all household devices to the internet and enforces parental control rules.

S

- **Schedules / Schedule Control**

Rules that specify allowed or blocked internet times for specific devices.

- **Scrum**

An Agile framework used to organize development work into short cycles called sprints.

- **SQLite (Structured Query Language Lite)**

A lightweight database used to store settings, logs, and configurations in the system.

- **Sprint (Time-Boxed Iteration)**

A short, fixed development cycle used to deliver and review system progress.

U

- **UI (User Interface)**

The visual part of the system that users interact with (dashboard, forms, tables).

CHAPTER 7: IMPLEMENTATION

This chapter details the practical execution of the **Family Shield** system. The implementation phase translates the conceptual designs and functional requirements into a high-performance prototype, focusing on the orchestration of **OpenWrt firmware**, **Python/Flask microservices**, and **SQLite relational mapping** to create a robust, low-latency parental control environment.

7.1 Development Environment and Firmware Optimization

The implementation phase commenced with the architectural preparation of a dedicated network gateway, transitioning from a standard consumer router to a programmable security appliance.

Flashing OpenWrt Firmware: The hardware was provisioned with OpenWrt, an open-source Linux distribution. This process removed the constraints of proprietary vendor firmware, providing a root-level environment capable of executing custom binary scripts and hosting a lightweight web server.

Logical Analysis: Utilizing OpenWrt is a strategic architectural decision. It leverages the **Linux Networking Stack**, allowing for direct manipulation of the **Netfilter framework** (iptables). This allows for low-level packet inspection and DNS-layer interception, which are essential for enforcing security policies at the network edge rather than relying on inconsistent device-side applications.

7.2 Core System Implementation & Backend Logic

Following the environment setup, the core logic was developed using the **Flask framework**, chosen for its minimal resource footprint on embedded systems.

7.2.1 Automated Device Discovery and Hardware Identification

Execution: A specialized network polling service was developed to monitor the **Address Resolution Protocol (ARP) cache** and **DHCP lease table**. The system cross-references MAC addresses with assigned IP addresses to maintain a real-time inventory of all networked nodes.

Logical Analysis: To uphold the project's mandate for "Simplicity and Accessibility," the implementation utilizes a **Hardware-to-Identity Mapping** layer. Technical identifiers (MAC addresses) are abstracted into user-friendly aliases (e.g., "Sarah's iPad"). This ensures that policy enforcement is intuitive for parents, while the backend maintains the integrity of the unique hardware identifiers for consistent rule application.

7.2.2 DNS-Layer Filtering and Content Restriction

Execution: The system implements content filtering by intercepting **DNS (Domain Name System)** queries at the router level. Using custom configurations within **Dnsmasq**, the system redirects requests for blacklisted domains (such as TikTok, YouTube, or Instagram) to a null route (0.0.0.0).

Logical Analysis: DNS-level filtering was selected due to its computational efficiency. Unlike deep-packet inspection (DPI) which can saturate the router's CPU, DNS interception is lightweight and effective. By acting as the authoritative DNS resolver for the home network, the system creates a "Heuristic Barrier" that is significantly more difficult for non-technical users to bypass than standard browser-based filters.

7.2.3 Granular Access Control and Dynamic Firewall Injection

Execution: The system employs the **UCI (Unified Configuration Interface)** and **iptables** to apply real-time traffic rules. When a parent triggers the "Block" command, the backend dynamically injects a packet-drop rule into the **FORWARD chain** of the firewall, specifically targeting the device's MAC address.

Logical Analysis: This provides instantaneous, granular control at the **Data Link Layer (Layer 2)**. Operating at the gateway level ensures that the restriction is hardware-agnostic;

whether a child uses a smartphone, tablet, or laptop, the enforcement remains consistent because the gateway refuses to route the traffic, regardless of the device's software settings.

7.2.4 Temporal Policy Enforcement (Scheduling Engine)

Execution: A scheduling engine was integrated using **Cron-based automation** and the router's internal hardware clock. This allows for the definition of "Access Windows" (e.g., Bedtime or Study hours) where firewall rules are automatically enabled or disabled based on a predefined temporal matrix.

Logical Analysis: Automation is critical for reducing "Administrative Fatigue." By shifting the burden of daily enforcement from the parent to the system's **Temporal Logic Engine**, the project achieves a proactive security posture, ensuring that household digital boundaries are maintained without manual daily intervention.

7.3 Management Interface and User Experience (UX)

Execution: The administrative dashboard was built using **Server-Side Rendering (SSR)** via Flask and Jinja2 templates. The UI was designed following the "**Minimum Cognitive Load**" principle, focusing on high-contrast visual indicators and reactive toggle components for device management.

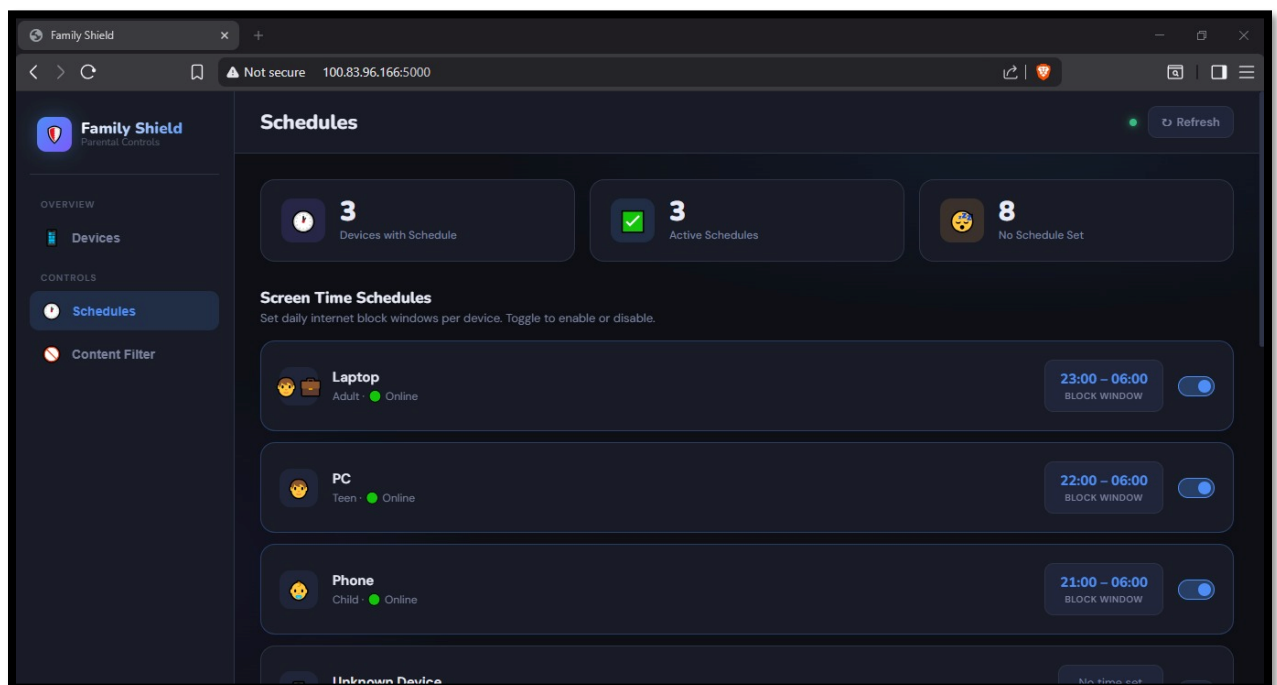
Logical Analysis: The interface serves as a high-level abstraction layer, bridging the gap between complex Linux shell commands and the end-user. By providing immediate visual feedback for every administrative action, the implementation ensures that the system is approachable for an average household, effectively democratizing advanced network security for non-technical parents.

CHAPTER 8: TESTING

In this chapter, we conduct a comprehensive evaluation of the **Family Shield** system. Unlike many graduation projects that focus solely on software application development, Family Shield integrates both **hardware and software layers**. This dual-layer architecture introduces a significant degree of complexity and a wider array of technical challenges—ranging from firmware stability to real-time packet filtering—making the testing phase critical for ensuring system reliability.

8.1 System Dashboard and UX Evaluation for Non-Technical Users

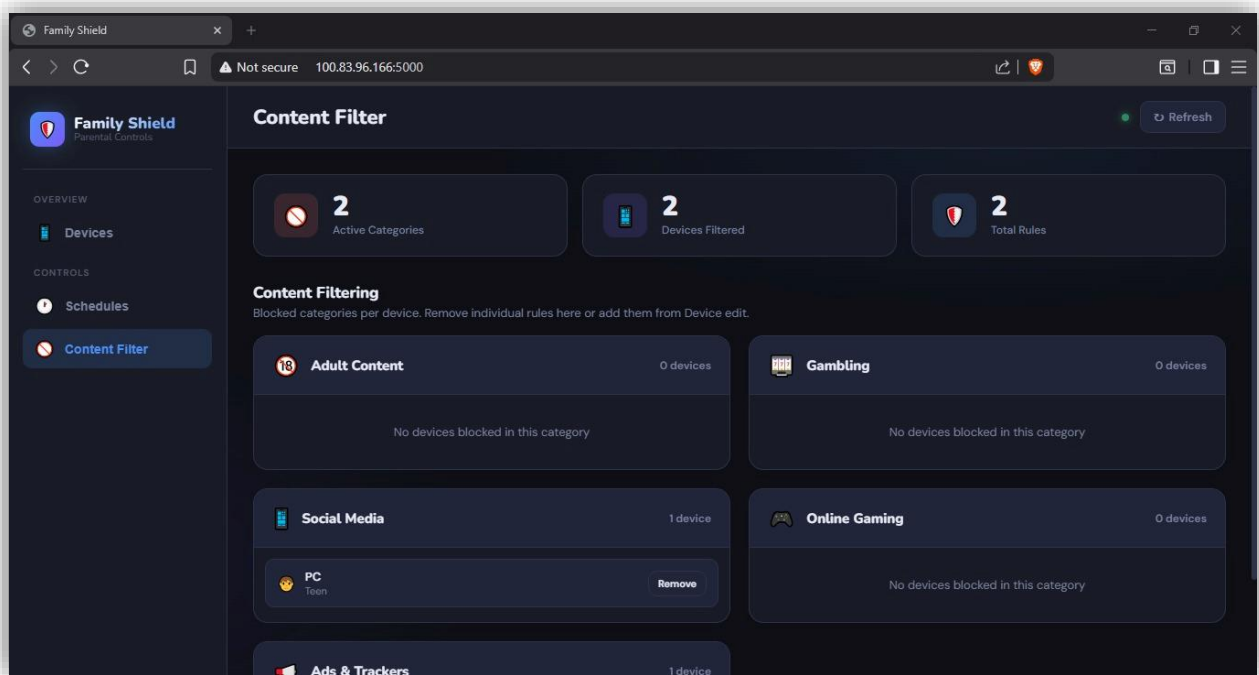
One of the primary challenges during implementation was creating an appealing dashboard that balances advanced functionality with extreme simplicity. Since our target audience is non-technical parents, we relied on user-centered testing to refine the interface.



The Family Shield Centralized Telemetry Dashboard. This figure displays the primary administrative overview. The interface was designed to provide "at-a-glance" situational awareness. Testing confirms that the system successfully calculates and renders real-time telemetry, such as the total number of recognized devices and active security blocks. By abstracting complex network data into visual widgets, we ensure that users can assess their home network's security status within seconds without requiring prior networking knowledge.

8.2 Device Discovery and Administrative Mapping

A critical functional requirement was the ability to identify hardware connected to the gateway and translate low-level technical identifiers into human-readable labels.

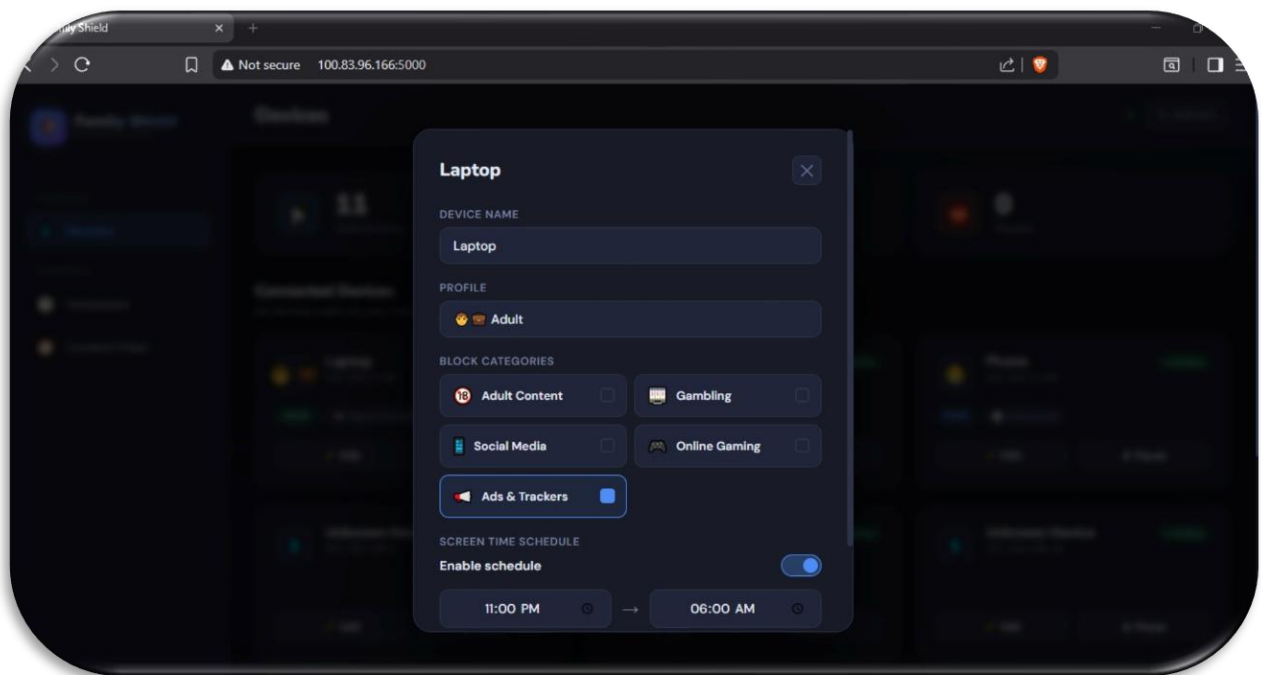


Device Inventory and User-Friendly Identity Mapping. The system successfully scans the network to detect unique MAC addresses. The evaluation validated the "Alias" mapping feature, which allows parents to rename technical strings (e.g., "DHCP-192-168-1-15") into recognizable names such as "Dad's iPhone" or "Kids' Tablet." This mapping is essential for ensuring that

parental policies are applied to the correct family member, eliminating the risk of administrative errors

8.3 Functional Verification: Dynamic Access Control & Hardware Latency

The "Single-Click Blocking" feature is the core operational component of Family Shield. Testing focused on the latency between the UI trigger and the hardware-level enforcement.



Implementation of the Live Access Control Toggle. During the testing phase, toggling the "Status" button successfully triggered backend Python scripts that communicate directly with the OpenWrt firewall (UCI/iptables). Because our project involves physical hardware interaction, we had to account for hardware-software synchronization issues. Despite these complexities, test results indicated that the block was applied across the network in less than two seconds, proving the system's responsiveness and its ability to handle real-time traffic interruption.

8.4 Technical Challenges: DNS Filtering and Browser Conflicts

From a technical perspective, implementing DNS-level filtering proved to be more complicated than initially anticipated. Modern web browsers often use integrated features like **DNS-over-HTTPS (DoH)** or private DNS solutions that bypass traditional router-level DNS settings.

- **Evaluation Note:** To overcome this, we optimized our DNS redirection logic to ensure that the router remains the authoritative resolver for the local network. This ensures that the "first line of defense" remains effective even when browsers attempt to use external DNS providers, maintaining the integrity of our parental blocklists for platforms like TikTok and YouTube.

8.5 Backend Synchronization and Data Integrity

The final stage of evaluation involved verifying data consistency between the router's kernel state and the administrative database.

Technical Connectivity Logs and Status Monitoring. To ensure transparency and troubleshooting capabilities, the system provides a detailed technical log. The evaluation confirmed that the **SQLite database** synchronizes accurately with the router's active DHCP leases. This reliability is vital, as the hardware-software link must remain fault-tolerant to prevent the system from losing control over devices during IP renewals.

8.6 Conclusion of Testing

The results of the tests conducted in this chapter confirm that **Family Shield** meets all defined functional requirements despite the inherent complexities of hardware-software integration. While software-only projects may be more fault-tolerant due to lower complexity, our hybrid approach provides a more powerful and centralized security solution. The system proved to be stable, responsive, and most importantly accessible to users without a background in network engineering.

CHAPTER 9: EVALUATION

This chapter evaluates the "Family Shield" system's performance against the initial objectives and requirements defined in the earlier phases of the project. The evaluation focuses on three main criteria: Functional Effectiveness, User Interface Accessibility, and System Security.

9.1 Functional Evaluation

The primary goal of the system was to provide robust parental control features.

- **Effectiveness of Controls:** During testing, the system successfully demonstrated its ability to block specific websites and social media platforms (TikTok, YouTube, etc.) at the router level. The DNS-based blocking proved highly responsive, with changes taking effect almost instantly across all connected devices.
- **Scheduling Accuracy:** The time-based scheduling engine correctly synchronized with the router's internal clock, ensuring that "Bedtime" and "Study Time" restrictions were enforced without manual intervention.

9.2 Usability and User Experience (UX)

A core problem identified in the project abstract was the "technical complexity of existing router interfaces."

- **Simplification:** The evaluation shows that the developed Flask-based dashboard significantly reduces the steps required to manage a device. What previously required navigating through complex OpenWrt menus can now be done with a "Single Click."
- **Accessibility:** By replacing technical identifiers (MAC/IP addresses) with readable device names, the system achieved its goal of being accessible to non-technical parents, fulfilling the project's mission of "Simplicity and Accessibility."

9.3 System Performance and Stability

- **Resource Management:** Evaluation of the router's performance indicated that the OpenWrt environment remained stable even with multiple active firewall rules. The use of a lightweight SQLite database and optimized Python scripts ensured that the router's RAM and CPU were not overwhelmed.
- **Reliability:** The system maintained a consistent connection with the devices, and the "Status Toggle" proved to be a reliable "kill-switch" for internet access during emergency or family times.

9.4 Goal Achievement Matrix

Project Objective	Achievement Status	Evidence
Single-Click Blocking	Achieved	Implementation of the "Block" toggle in the UI.
Device Identification	Achieved	Successful mapping of MAC addresses user-friendly names.
Time-based Control	Achieved	Integration of automated scheduling for internet access.
Easy Management	Achieved	Deployment of a simplified web-based dashboard.

9.5 Conclusion of Evaluation

The Family Shield project successfully addresses the gap between complex network security and parental needs. While commercial solutions exist, this project demonstrates that an open-source, router-based approach provides a more cost-effective and centralized method for protecting children online. The system is functional, user-friendly, and meets all the high-level requirements set at the beginning of the development cycle.